

COMP4616 – Fundamentals of Machine Learning

ASST. PROF. TUĞBA ERKOÇ

SPRING 2026

WEEK 3: DECISION TREES



Recap

- Decision trees classify by following tests from root to leaf
- ID3 builds the tree top-down
- At each step, choose the attribute with the highest information gain
- Entropy measures impurity
- Information gain measures reduction in entropy after a split

Practical Implementation of Decision Trees

- In theoretical examples, we often use
 - Entropy
 - Information Gain
 - ID3
- In practical ML libraries, decision trees often implemented with
 - Gini impurity
 - CART (Classification and Regression Trees) approach

Practical Implementation of Decision Trees

CART style splitting

- Algorithm always creates two branches at each split
 - In other words, it performs a binary split
- **Example:** Attribute Age
 - Instead of creating many branches like
 - $\text{Age} < 20$
 - $20 \leq \text{Age} < 40$
 - $\text{Age} \geq 40$
 - CART will create a binary decision
 - $\text{Age} \leq 30$
 - $\text{Age} > 30$
- The algorithm tries many possible split points and chooses the one that produces the **purest groups of data**
- To measure how pure the resulting groups are, CART typically uses a metric called **Gini impurity** instead of entropy

Entropy vs Gini impurity

- Entropy and Gini Measure the Same Idea
- Both entropy and Gini are impurity measures.
- If a node is pure, both become zero.

$$H(S) = - \sum_i p_i \log_2 p_i$$
$$Gini(S) = 1 - \sum_i p_i^2$$

Distribution	Entropy	Gini
Pure node (1,0)	0	0
Balanced (0.5,0.5)	High	High
Mostly one class (0.8,0.2)	Lower	Lower

What is generalization?

- A model should fit the training data
- But it must also perform well on unseen data
 - A machine learning model is not useful just because it memorizes training examples
- Good generalization = good performance on test data
- We evaluate generalization using a separate **test set**
- A hypothesis **generalizes well** if it predicts unseen examples accurately

Overfitting and Underfitting

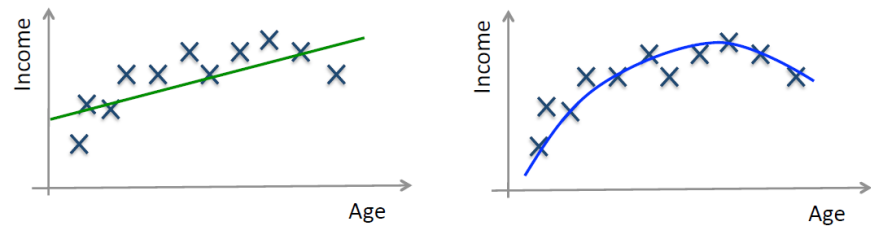
- A model is **underfitting** when it is too simple to capture the important patterns in the data
- A model is **overfitting** when it learns the training data too specifically, including noise or accidental details, so its performance on unseen data becomes poor
- Overfitting becomes more likely when
 - the model is too complex (for example **larger decision trees**)
 - the dataset is small
 - the number of features is large
- Overfitting can become less likely when we have more training data

Bias and Variance

- **Bias** is the difference between the average prediction of our model and the correct value which we are trying to predict.
- **High bias (underfitting)** means the model is too simple and tends to miss the true pattern
- **Variance** measures how sensitive the model is to the particular training set
- **High variance (overfitting)** means the model depends too much on the particular training set and may not generalize well

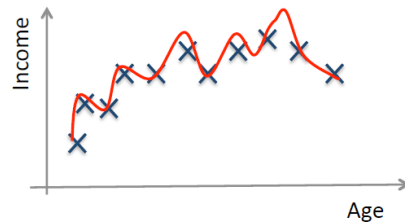
Model 1: Training Error = 2% Testing Error = 18%	⇒ "Low Bias, High Variance". ★ Over-Fitting ★
Model 2: Training Error = 24% Testing Error = 28%	⇒ "High Bias, High Variance" ★ Under-Fitting ★
Model 3: Training Error = <10% Testing Error = <10%	⇒ "Low Bias, Low Variance" ★ Most Generalized Model ★ → Recommended!

Underfitting and Overfitting

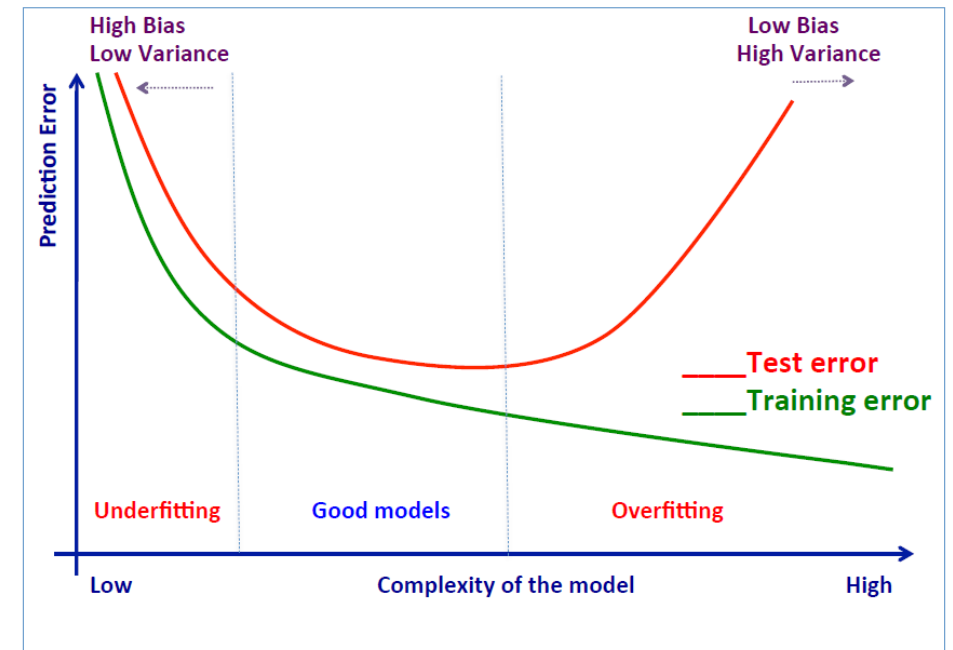


High bias (underfitting)

Just right!

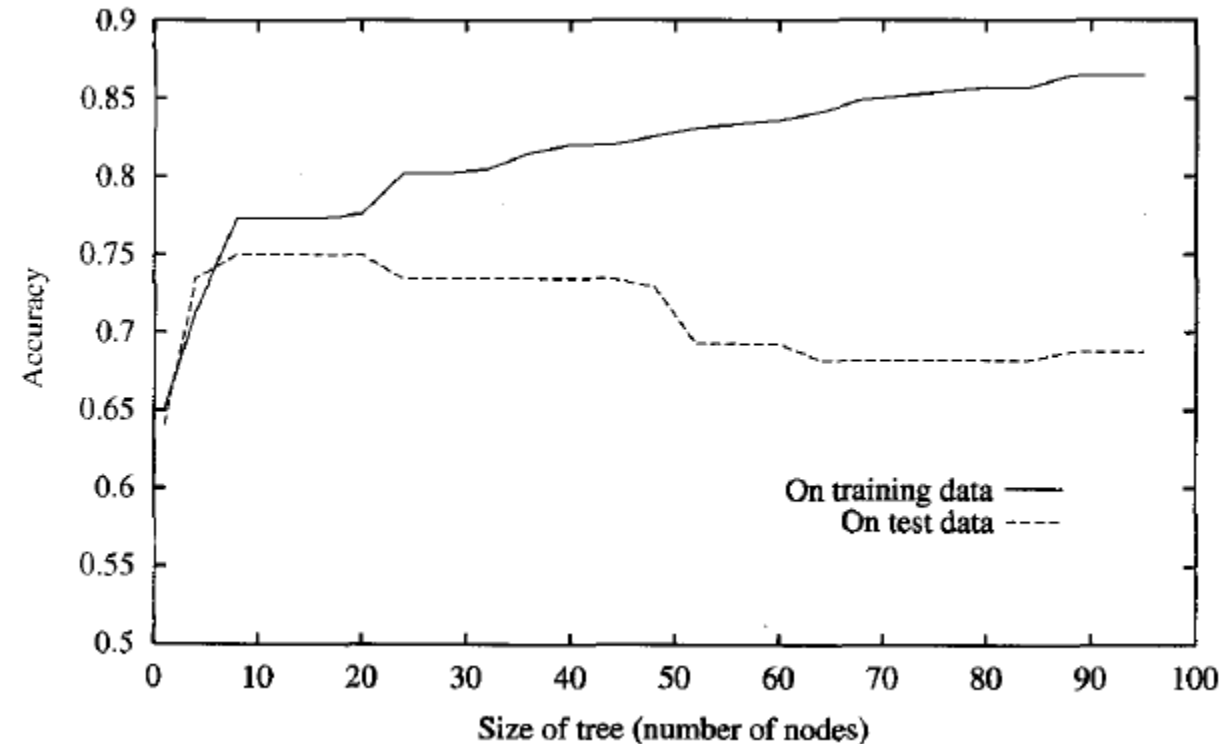


High variance (overfitting)



What Happens as Tree Size Grows?

- As tree size increases
 - training accuracy usually keeps increasing
 - test accuracy often increases at first
 - then test accuracy may decrease
 - More nodes = more capacity to fit data
 - More capacity also means more risk of overfitting
- In this example, when the tree size exceeds approximately 25 nodes, further elaboration of the tree decreases its accuracy over the test examples despite increasing its accuracy on the training examples

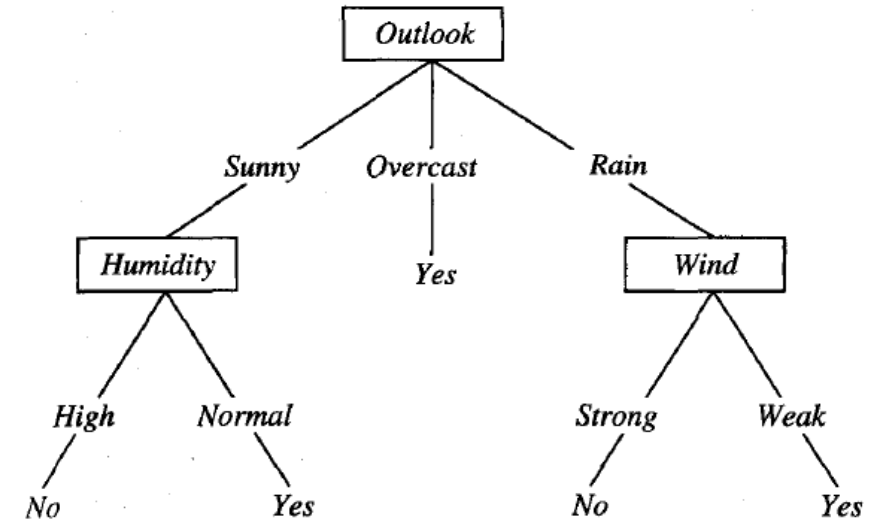


Why decision trees overfit easily?

- Trees split recursively
- If allowed to grow too much, they can memorize small details
- They can create branches for
 - Outliers
 - mislabeled examples
 - random noise
- This produces complex trees with poor generalization

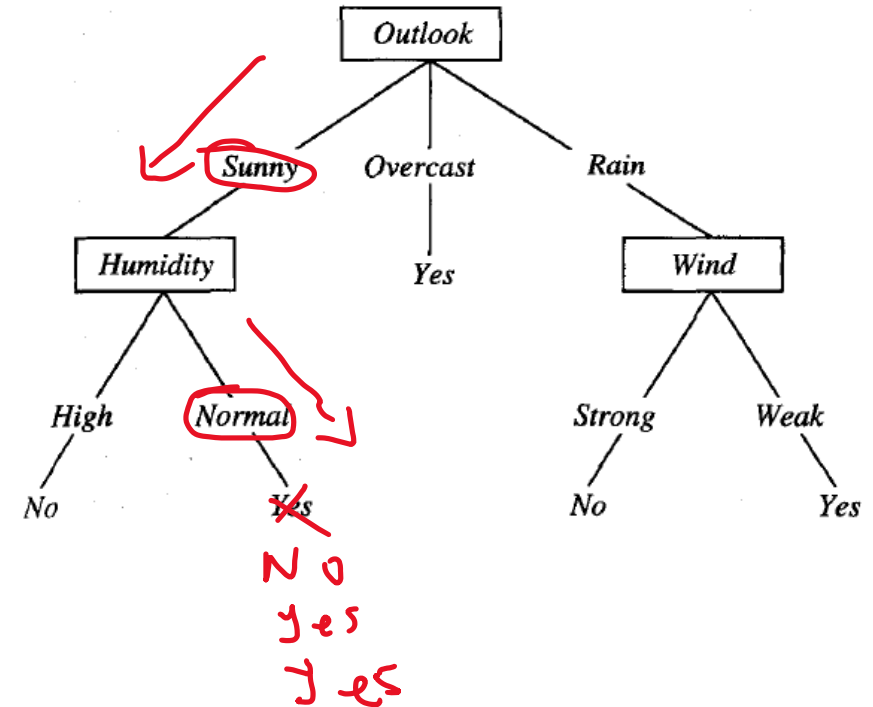
Why decision trees overfit easily?

- Example
 - Training examples contain random errors or noise
- Let's consider the PlayTennis example.
- Assume that we have this example with incorrectly labelled as negative:
 - $\langle \text{Outlook} = \text{Sunny}, \text{Temperature} = \text{Hot}, \text{Humidity} = \text{Normal}, \text{Wind} = \text{Strong}, \text{PlayTennis} = \text{No} \rangle$
- Without this error, we would produce the tree on the right side.
- The addition of this incorrect example will now cause ID3 to construct a more complex tree.



Generalization and Overfitting

- The new example will be sorted into the second leaf node from the left in the learned tree along with the previous positive examples D9 and D11.
- Because the new example is incorrectly labeled as a negative example, ID3 will search for further refinements to the tree below this node.
- ID3 will split this node further via an appropriate attribute, so we will get more complex tree.
- In one experimental study of ID3 involving five different learning tasks with noisy, nondeterministic data (Mingers 1989b), overfitting was found to decrease the accuracy of learned decision trees by 10-25% on most problems



How to Avoid Overfitting

- There are several approaches to avoiding overfitting in decision tree learning. These can be grouped into two classes
 - Pre-pruning
 - Post-pruning
- Pre-pruning approaches stop growth before the tree fully fits the training data
- Post-pruning approaches allow the tree to overfit the data, and then removes branches that do not improve generalization
- Although the first of these approaches might seem more direct, the second approach of post-pruning overfit trees has been found to be more successful in practice.
 - This is due to the difficulty in the first approach of estimating precisely when to stop growing the tree.

Pre-pruning: Stop the Tree Early

- Stop growth before the tree fully fits the training data

- **Examples**

- limit depth
 - require enough samples to split
 - require minimum samples in leaves

- **Advantage**

- simpler, faster model

- **Risk**

- may stop too early and miss important patterns

Hyperparameters that control tree complexity

- In scikit-learn, there are hyperparameters that can control the tree complexity
 - **max_depth**: maximum depth of the tree
 - **min_samples_leaf**: minimum number of training samples that must remain in a leaf node
- How depth is counted?
 - root node is at depth 0
 - its children are at depth 1
 - next level is depth 2
 - and so on
- max_depth=1 → root + one split
- max_depth=2 → one more layer of splits
- max_depth=None → no explicit depth limit

Hyperparameters that control tree complexity

- **small max_depth**

- simpler model
- less likely to overfit
- may underfit

- **large max_depth**

- more flexible model
- can fit training data very well
- more likely to overfit

Hyperparameters that control tree complexity

- **min_samples_leaf** sets the minimum number of training samples that must remain in a leaf node
- Without restrictions, decision tree algorithm may create very small leaves that contain only 1 or 2 samples. That can make the tree too specific to the training set.
- **Smaller values**
 - allows tiny leaves
 - Can increase variance
 - tree can become very specific
 - higher risk of overfitting - very small leaves often mean the tree is memorizing details
- **Larger values**
 - forces leaves to contain more samples
 - tree becomes more conservative
 - can improve generalization

Post-pruning: Grow First, Then Simplify

- First build a large tree
- Then remove branches that do not improve generalization
- Often works better in practice than stopping too early
- Can be guided by
 - validation data
 - pruning criteria

Reduced-error pruning

- Start with a full tree
- Consider one internal node
- Replace its subtree with a leaf (assign the most common classification of the training examples affiliated with that node)
- Check validation accuracy
- Keep the pruning only if performance does not get worse

Rule Post-Pruning

- Grow the full decision tree first
 - This tree may be **very large and overfit**
- Convert the tree into a set of if-then rules
 - Each **root to leaf path** becomes a rule
 - IF Outlook = Sunny AND Humidity = High THEN PlayTennis = No
- Simplify each rule by removing unnecessary conditions
 - Try removing one condition
 - IF Humidity = High THEN PlayTennis = No, or
 - IF Outlook = Sunny THEN PlayTennis = No
- Keep the simpler rule if it improves performance or does not reduce accuracy

